1. Implement Tic –Tac –Toe Game

Implement "Tic-tac-Toe" in general without using any standard algorithm.

```
#Tic-Tac-Toe Game
# Human = X
# Computer = O
board = [' ' for _ in range(9)]
def display_board():
    print()
    print(" ", board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(" ", board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(" ", board[6], "|", board[7], "|", board[8])
    print()
def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == player and board[b] == player and board[c] == player:
            return True
    return False
def board_full():
    return ' ' not in board
def computer_move():
    # First try to win
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            if check_winner('O'):
                return
            board[i] = ' '
    # Then block the human
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            if check_winner('X'):
                board[i] = 'O'
                return

            board[i] = ' '
```

```
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            return
print("TIC-TAC-TOE")
print("Human = X")
print("Computer = O")
while True:
    display_board()
    # Human move
    while True:
        try:
            position = int(input("Enter your position (1-9): "))

            if position < 1 or position > 9:
                print("Enter a number between 1 and 9.")
            elif board[position - 1] != ' ':
                print("Position already occupied.")
            else:
                board[position - 1] = 'X'
                break
        except ValueError:
            print("Please enter a valid number.")

    if check_winner('X'):
        display_board()
        print("Human wins!")
        break

    if board_full():
        display_board()
        print("Game Draw!")
        break
    # Computer move
    computer_move()

    print("Computer made its move.")

    if check_winner('O'):
        display_board()
        print("Computer wins!")
        break

    if board_full():
        display_board()
        print("Game Draw!")
        break
```

OUTPUT:

```
PS C:\Users\BMSCE\Desktop\Vinay-AI> & C:\Users\BMSCE\AppData\Local
TIC-TAC-TOE
Human = X
Computer = O

   |   |
---+---+---
   |   |
---+---+---
   |   |

Enter your position (1-9): 5
Computer made its move.

  O |   |
---+---+---
   | X |
---+---+---
   |   |

Enter your position (1-9): & C:\Users\BMSCE\AppData\Local\Programs
Please enter a valid number.
Enter your position (1-9): 9
Computer made its move.

  O | O |
---+---+---
   | X |
---+---+---
   |   | X

Enter your position (1-9): 3
Computer made its move.

  O | O | X
---+---+---
   | X | O
---+---+---
   |   | X

Enter your position (1-9): 7

  O | O | X
---+---+---
   | X | O
---+---+---
  X |   | X

Human wins!
```

2. Implement vacuum cleaner agent Implement "Vacuum Cleaner Agent" using the concept of "Simple Reflex agent and Goal based" architecture.

```
# Vacuum Cleaner Agent

def simple_reflex_agent(location, status):
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Move Right"
    else:
        return "Move Left"


def goal_based_agent(location, room_a, room_b):
    actions = []

    while room_a == "Dirty" or room_b == "Dirty":

        if location == "A":
            if room_a == "Dirty":
                actions.append("Suck")
                room_a = "Clean"
            else:
                actions.append("Move Right")
                location = "B"

        else:
            if room_b == "Dirty":
                actions.append("Suck")
                room_b = "Clean"
            else:
                actions.append("Move Left")
                location = "A"

    actions.append("Goal Achieved")
    return actions


# Simple Reflex Agent

print("----- SIMPLE REFLEX AGENT -----")

location = input("Enter vacuum location (A/B): ")
status = input("Enter status (Clean/Dirty): ")

action = simple_reflex_agent(location, status)

print("Action:", action)
```

```python
# Goal Based Agent

print("\n----- GOAL BASED AGENT -----")

location = input("Enter vacuum location (A/B): ")
room_a = input("Enter status of Room A (Clean/Dirty): ")
room_b = input("Enter status of Room B (Clean/Dirty): ")

actions = goal_based_agent(location, room_a, room_b)

print("\nActions:")
for action in actions:
    print(action)
```

OUTPUT:

```
PS C:\Users\BMSCE\Desktop\Vinay-AI> & C:\Users\BMSCE\AppData\Local
----- SIMPLE REFLEX AGENT -----
Enter vacuum location (A/B): A
Enter status (Clean/Dirty): Clean
Action: Move Right

----- GOAL BASED AGENT -----
Enter vacuum location (A/B): A
Enter status of Room A (Clean/Dirty): Clean
Enter status of Room B (Clean/Dirty): Dirty

Actions:
Move Right
Suck
Goal Achieved
```